Bayesian Model for Cognitive Decline Prediction

Manvi, Annika, Ruby

April 11, 2026

Outline

We're applying Bayesian linear regression to longitudinal MMSE scores from the NACC dataset to predict how a patient's cognitive function might change over time. For each patient, we model MMSE as a linear function of age and put a conjugate prior on the regression weights. Then we run a Gibbs sampler to get the posterior and make predictions for future visits.

Data Preparation

```
con <- unz("/Users/kkmanas/ds4420-finalproject/data/NACC_visitation_data.zip",
           "investigator_ftldlbd_nacc72.csv")
df_raw <- read.csv(con)

df <- df_raw[, c("NACCID", "NACCAGE", "NACCMMSE")]

sentinel_vals <- c(88, 95, 96, 97, 98, -4)
df$NACCMMSE[df$NACCMMSE %in% sentinel_vals] <- NA
df <- df[!is.na(df$NACCMMSE) & !is.na(df$NACCAGE), ]

cat("Rows after cleaning:", nrow(df), "\n")
```

```
## Rows after cleaning: 99470
```

```
cat("Unique patients:    ", length(unique(df$NACCID)), "\n")
```

```
## Unique patients:     31790
```

```
visit_counts <- table(df$NACCID)

# Train/test split helper (80/20)
split_patient <- function(sub) {
  n <- nrow(sub)
  n_tr <- floor(n * 0.8)
  sub$split <- c(rep("train", n_tr), rep("test", n - n_tr))
  sub
}
```

Select the best threshold

We want to figure out the right minimum number of visits to require. If the threshold is too low, we won't have enough data per patient, and if it's too high, we'll lose too many patients. We'll loop over thresholds from 4 to 8 visits, run the full Gibbs sampler for each, and select the one with the lowest RMSE on the test visits.

```r
library(MCMCpack)
library(mvnfast)

# Create Gibbs sampler to test across thresholds
gibbs_patient_fn <- function(X_tr, y_tr, m0, V0, alpha0, beta0,
                             no_samples = 2000, n_warmup = 500) {
  n <- length(y_tr)
  V0_inv <- solve(V0)
  gibbs_samples_w <- matrix(0, nrow = no_samples, ncol = 2)
  gibbs_samples_sigma2 <- matrix(0, nrow = no_samples, ncol = 1)
  w_tilde <- matrix(m0, ncol = 1)
  sigma2_tilde <- 1

  for (i in seq_len(no_samples)) {
    V <- solve((1 / sigma2_tilde) * t(X_tr) %*% X_tr + V0_inv)
    M <- V %*% ((1 / sigma2_tilde) * t(X_tr) %*% y_tr + V0_inv %*% m0)
    w_tilde <- t(rmvn(1, M, V))
    A <- (2 * alpha0 + n) / 2
    B <- (2 * beta0 + t(y_tr - X_tr %*% w_tilde) %*%
                  (y_tr - X_tr %*% w_tilde)) / 2
    sigma2_tilde <- rinvgamma(1, A, B)
    gibbs_samples_w[i, ]      <- w_tilde
    gibbs_samples_sigma2[i, ] <- sigma2_tilde
  }
  idx <- (n_warmup + 1):no_samples
  cbind(w0 = gibbs_samples_w[idx, 1], w1 = gibbs_samples_w[idx, 2],
        sigma2 = gibbs_samples_sigma2[idx, 1])
}

# Compare across thresholds
thresholds <- c(4, 5, 6, 7, 8)
threshold_results <- data.frame(
  min_visits = thresholds,
  n_patients = NA, rmse = NA, coverage = NA
)

for (ti in seq_along(thresholds)) {
  thresh <- thresholds[ti]

  # Filter and split
  ids <- names(visit_counts[visit_counts >= thresh])
  dm <- df[df$NACCID %in% ids, ]
  dm <- dm[order(dm$NACCID, dm$NACCAGE), ]
  ds <- do.call(rbind, lapply(split(dm, dm$NACCID), split_patient))
  dtr <- ds[ds$split == "train", ]
  dte <- ds[ds$split == "test",  ]
```

```r
# Empirical Bayes prior - fit least squares, then extract both coefs and residual var
ols_t <- lapply(ids, function(id) {
  s <- dtr[dtr$NACCID == id, ]
  if (nrow(s) < 3) return(NULL)
  lm(NACCMMSE ~ NACCAGE, data = s)
})
ols_t <- ols_t[!sapply(ols_t, is.null)]
coefs_t <- do.call(rbind, lapply(ols_t, coef))
coefs_t <- coefs_t[complete.cases(coefs_t), ]
m0_t <- colMeans(coefs_t)
V0_t <- 5 * diag(2)
rv <- mean(sapply(ols_t, function(fit) {
  v <- var(residuals(fit))
  if (is.na(v) || v == 0) return(NA)
  v
}), na.rm = TRUE)
a0 <- 3
b0 <- a0 * rv

# Run Gibbs for all patients at this threshold
samps <- vector("list", length(ids))
names(samps) <- ids
cat("Threshold", thresh, "- running Gibbs for", length(ids), "patients...\n")
for (j in seq_along(ids)) {
  id <- ids[j]
  s  <- dtr[dtr$NACCID == id, ]
  if (nrow(s) < 2) next
  samps[[id]] <- gibbs_patient_fn(cbind(1, s$NACCAGE), s$NACCMMSE,
                                  m0_t, V0_t, a0, b0)
  if (j %% 200 == 0)
    cat("  ", j, "/", length(ids), "done\n")
}

# Posterior predictive on test set
pp_list <- lapply(ids, function(id) {
  st <- dte[dte$NACCID == id, ]
  d  <- samps[[id]]
  if (nrow(st) == 0 || is.null(d)) return(NULL)
  idx <- sample(nrow(d), 1000, replace = TRUE)
  X_new <- cbind(1, st$NACCAGE)
  pm <- matrix(NA, 1000, nrow(st))
  for (k in seq_along(idx)) {
    w <- d[idx[k], c("w0","w1")]; s2 <- d[idx[k], "sigma2"]
    pm[k, ] <- rnorm(nrow(st), X_new %*% w, sqrt(s2))
  }
  data.frame(NACCID = id, NACCAGE = st$NACCAGE,
             pp_mean  = colMeans(pm),
             pp_lower = apply(pm, 2, quantile, 0.055),
             pp_upper = apply(pm, 2, quantile, 0.945))
})
pp_df <- do.call(rbind, Filter(Negate(is.null), pp_list))
eval <- merge(dte[, c("NACCID","NACCAGE","NACCMMSE")],
              pp_df, by = c("NACCID","NACCAGE"))
```

```r
  threshold_results$n_patients[ti] <- length(ids)
  threshold_results$rmse[ti] <- sqrt(mean((eval$NACCMMSE - eval$pp_mean)^2))
  threshold_results$coverage[ti] <- mean(eval$NACCMMSE >= eval$pp_lower &
                                          eval$NACCMMSE <= eval$pp_upper)
  cat("Threshold", thresh, "| Patients:", length(ids),
      "| RMSE:", round(threshold_results$rmse[ti], 3),
      "| Coverage:", round(threshold_results$coverage[ti] * 100, 1), "%\n")
}
```

```
## Threshold 4 - running Gibbs for 10329 patients...
##    200 / 10329 done
##    400 / 10329 done
##    600 / 10329 done
##    800 / 10329 done
##    1000 / 10329 done
##    1200 / 10329 done
##    1400 / 10329 done
##    1600 / 10329 done
##    1800 / 10329 done
##    2000 / 10329 done
##    2200 / 10329 done
##    2400 / 10329 done
##    2600 / 10329 done
##    2800 / 10329 done
##    3000 / 10329 done
##    3200 / 10329 done
##    3400 / 10329 done
##    3600 / 10329 done
##    3800 / 10329 done
##    4000 / 10329 done
##    4200 / 10329 done
##    4400 / 10329 done
##    4600 / 10329 done
##    4800 / 10329 done
##    5000 / 10329 done
##    5200 / 10329 done
##    5400 / 10329 done
##    5600 / 10329 done
##    5800 / 10329 done
##    6000 / 10329 done
##    6200 / 10329 done
##    6400 / 10329 done
##    6600 / 10329 done
##    6800 / 10329 done
##    7000 / 10329 done
##    7200 / 10329 done
##    7400 / 10329 done
##    7600 / 10329 done
##    7800 / 10329 done
##    8000 / 10329 done
##    8200 / 10329 done
##    8400 / 10329 done
##    8600 / 10329 done
```

```
##    8800 / 10329 done
##    9000 / 10329 done
##    9200 / 10329 done
##    9400 / 10329 done
##    9600 / 10329 done
##    9800 / 10329 done
##    10000 / 10329 done
##    10200 / 10329 done
## Threshold 4 | Patients: 10329 | RMSE: 3.79 | Coverage: 63.2 %
## Threshold 5 - running Gibbs for 7235 patients...
##    200 / 7235 done
##    400 / 7235 done
##    600 / 7235 done
##    800 / 7235 done
##    1000 / 7235 done
##    1200 / 7235 done
##    1400 / 7235 done
##    1600 / 7235 done
##    1800 / 7235 done
##    2000 / 7235 done
##    2200 / 7235 done
##    2400 / 7235 done
##    2600 / 7235 done
##    2800 / 7235 done
##    3000 / 7235 done
##    3200 / 7235 done
##    3400 / 7235 done
##    3600 / 7235 done
##    3800 / 7235 done
##    4000 / 7235 done
##    4200 / 7235 done
##    4400 / 7235 done
##    4600 / 7235 done
##    4800 / 7235 done
##    5000 / 7235 done
##    5200 / 7235 done
##    5400 / 7235 done
##    5600 / 7235 done
##    5800 / 7235 done
##    6000 / 7235 done
##    6200 / 7235 done
##    6400 / 7235 done
##    6600 / 7235 done
##    6800 / 7235 done
##    7000 / 7235 done
##    7200 / 7235 done
## Threshold 5 | Patients: 7235 | RMSE: 3.616 | Coverage: 70.7 %
## Threshold 6 - running Gibbs for 5206 patients...
##    200 / 5206 done
##    400 / 5206 done
##    600 / 5206 done
##    800 / 5206 done
##    1000 / 5206 done
##    1200 / 5206 done
```

```
##    1400 / 5206 done
##    1600 / 5206 done
##    1800 / 5206 done
##    2000 / 5206 done
##    2200 / 5206 done
##    2400 / 5206 done
##    2600 / 5206 done
##    2800 / 5206 done
##    3000 / 5206 done
##    3200 / 5206 done
##    3400 / 5206 done
##    3600 / 5206 done
##    3800 / 5206 done
##    4000 / 5206 done
##    4200 / 5206 done
##    4400 / 5206 done
##    4600 / 5206 done
##    4800 / 5206 done
##    5000 / 5206 done
##    5200 / 5206 done
## Threshold 6 | Patients: 5206 | RMSE: 3.447 | Coverage: 75.7 %
## Threshold 7 - running Gibbs for 3619 patients...
##    200 / 3619 done
##    400 / 3619 done
##    600 / 3619 done
##    800 / 3619 done
##    1000 / 3619 done
##    1200 / 3619 done
##    1400 / 3619 done
##    1600 / 3619 done
##    1800 / 3619 done
##    2000 / 3619 done
##    2200 / 3619 done
##    2400 / 3619 done
##    2600 / 3619 done
##    2800 / 3619 done
##    3000 / 3619 done
##    3200 / 3619 done
##    3400 / 3619 done
##    3600 / 3619 done
## Threshold 7 | Patients: 3619 | RMSE: 3.135 | Coverage: 78.7 %
## Threshold 8 - running Gibbs for 2327 patients...
##    200 / 2327 done
##    400 / 2327 done
##    600 / 2327 done
##    800 / 2327 done
##    1000 / 2327 done
##    1200 / 2327 done
##    1400 / 2327 done
##    1600 / 2327 done
##    1800 / 2327 done
##    2000 / 2327 done
##    2200 / 2327 done
## Threshold 8 | Patients: 2327 | RMSE: 2.881 | Coverage: 81.1 %
```

```r
print(threshold_results)
```

```
##   min_visits n_patients     rmse  coverage
## 1          4      10329 3.789678 0.6323151
## 2          5       7235 3.616264 0.7066172
## 3          6       5206 3.446665 0.7568656
## 4          7       3619 3.134784 0.7871318
## 5          8       2327 2.881432 0.8109861
```

```r
# Pick the threshold with the lowest RMSE
best_thresh <- threshold_results$min_visits[which.min(threshold_results$rmse)]
cat("\nBest threshold: >=", best_thresh, "visits\n")
```

```
##
## Best threshold: >= 8 visits
```

```r
# Note: update file path if needed
BAYES_LINEAR_THRESH_FILE <- "/Users/kkmanas/ds4420-finalproject/bayesian/bayes_linear_threshold_results

if (file.exists(BAYES_LINEAR_THRESH_FILE)) {
  threshold_results <- readRDS(BAYES_LINEAR_THRESH_FILE)
  best_thresh <- threshold_results$min_visits[which.min(threshold_results$rmse)]
  cat("Loaded saved BLR threshold results.\n")
} else {
  saveRDS(threshold_results, BAYES_LINEAR_THRESH_FILE)
  cat("BLR threshold results saved.\n")
}
```

```
## Loaded saved BLR threshold results.
```

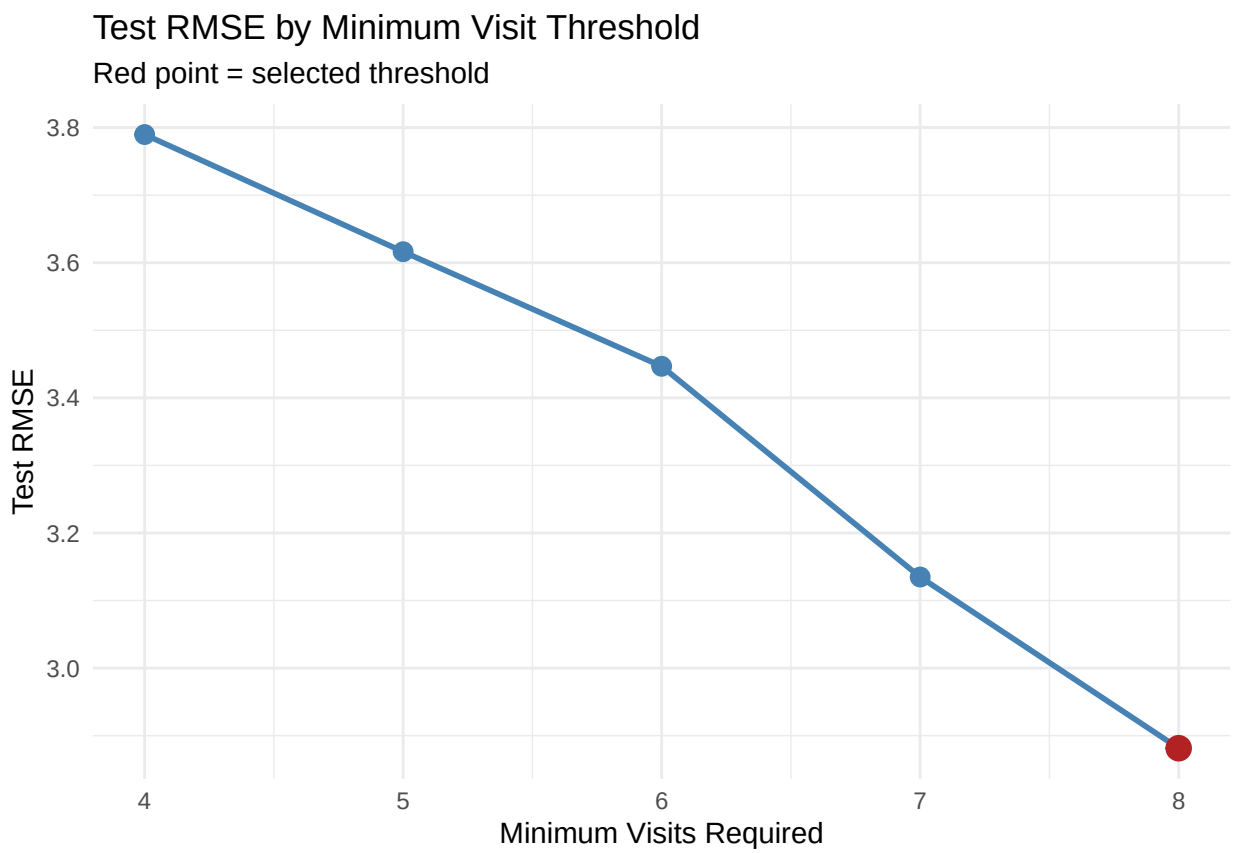
```r
print(threshold_results)
```

```
##   min_visits n_patients     rmse  coverage
## 1          4      10329 3.789678 0.6323151
## 2          5       7235 3.616264 0.7066172
## 3          6       5206 3.446665 0.7568656
## 4          7       3619 3.134784 0.7871318
## 5          8       2327 2.881432 0.8109861
```

```r
cat("Best threshold: >=", best_thresh, "visits\n")
```

```
## Best threshold: >= 8 visits
```

```r
library(ggplot2)
ggplot(threshold_results, aes(x = min_visits)) +
  geom_line(aes(y = rmse), color = "steelblue", linewidth = 1) +
  geom_point(aes(y = rmse), color = "steelblue", size = 3) +
  geom_point(data = threshold_results[which.min(threshold_results$rmse), ],
             aes(y = rmse), color = "firebrick", size = 4) +
  labs(title = "Test RMSE by Minimum Visit Threshold",
       subtitle = "Red point = selected threshold",
       x = "Minimum Visits Required", y = "Test RMSE") +
  theme_minimal()
```




```r
# Load threshold from file
if (!exists("best_thresh")) {
  threshold_results <- readRDS("/Users/kkmanas/ds4420-finalproject/bayesian/bayes_linear_threshold_resul
  best_thresh <- threshold_results$min_visits[which.min(threshold_results$rmse)]
}

# Apply the winning threshold for the rest of the model
patient_ids <- names(visit_counts[visit_counts >= best_thresh])
df_multi    <- df[df$NACCID %in% patient_ids, ]
df_multi    <- df_multi[order(df_multi$NACCID, df_multi$NACCAGE), ]
df_split    <- do.call(rbind, lapply(split(df_multi, df_multi$NACCID), split_patient))
df_train    <- df_split[df_split$split == "train", ]
df_test     <- df_split[df_split$split == "test",  ]

cat("Selected threshold: >=", best_thresh, "visits\n")
```

```
## Selected threshold: >= 8 visits
```

```r
cat("Patients:              ", length(patient_ids), "\n")
```

```
## Patients:               2327
```

```r
cat("Training observations:", nrow(df_train), "\n")
```

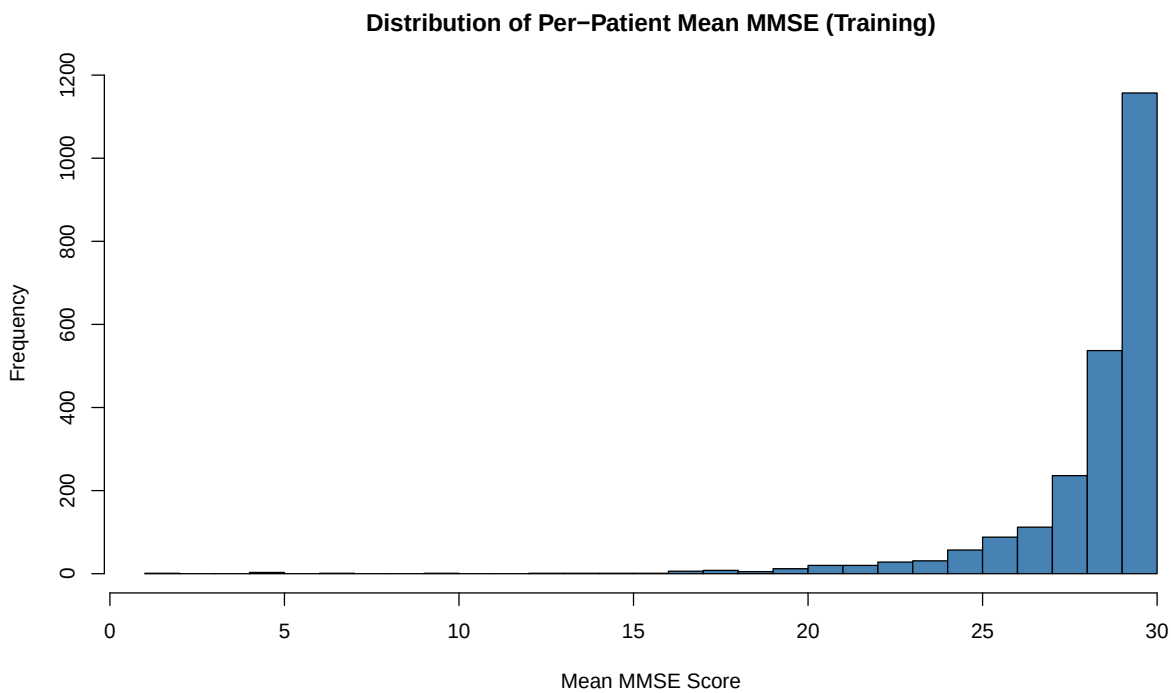
```
## Training observations: 16544
```

```r
cat("Test observations:    ", nrow(df_test),  "\n")
```

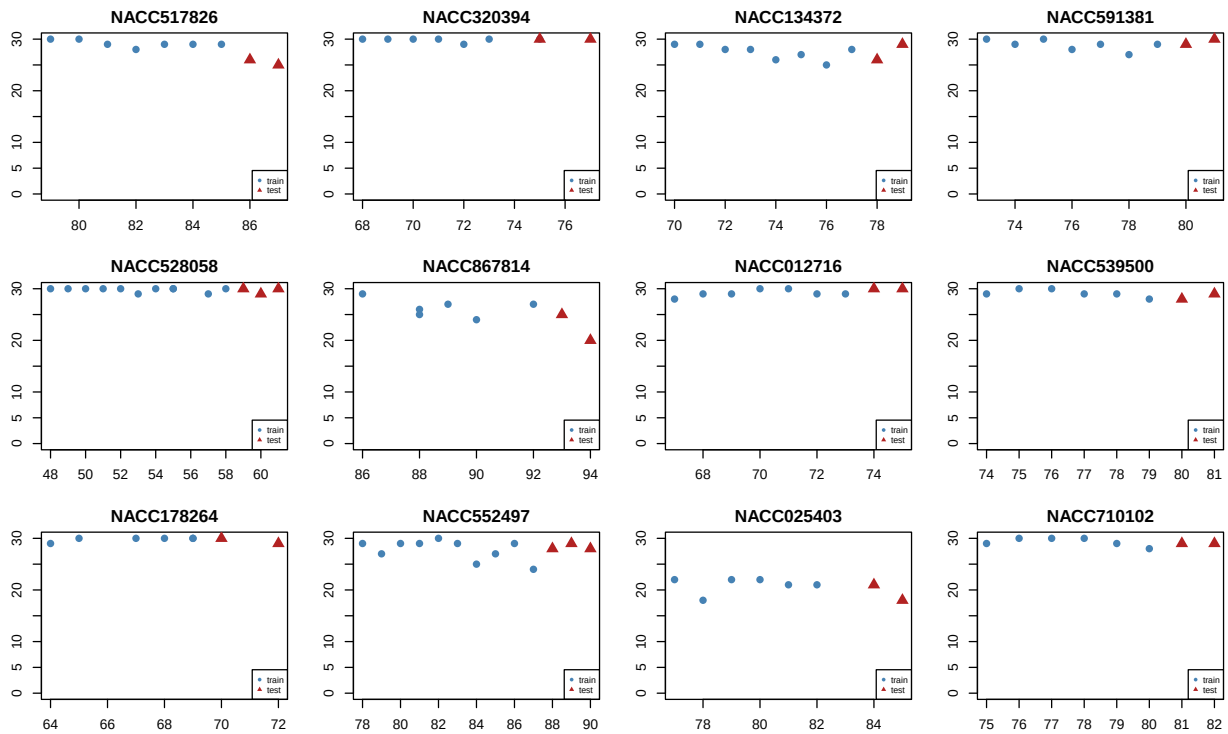
```
## Test observations:     5081
```

Exploratory Data Analysis

```r
# Distribution of mean MMSE per patient (training data)
mean_mmse <- tapply(df_train$NACCMMSE, df_train$NACCID, mean)
hist(mean_mmse, breaks = 40, col = "steelblue",
     main = "Distribution of Per-Patient Mean MMSE (Training)",
     xlab = "Mean MMSE Score")
```




```r
# Raw data for 12 patients (as an example)
example_ids <- sample(patient_ids, 12)
par(mfrow = c(3, 4), mar = c(3, 3, 2, 1))
for (id in example_ids) {
  sub <- df_split[df_split$NACCID == id, ]
  tr  <- sub[sub$split == "train", ]
  te  <- sub[sub$split == "test",  ]
  plot(sub$NACCAGE, sub$NACCMMSE, type = "n",
       main = id, xlab = "Age", ylab = "MMSE", ylim = c(0, 30))
  points(tr$NACCAGE, tr$NACCMMSE, pch = 16, col = "steelblue", cex = 1.2)
  points(te$NACCAGE, te$NACCMMSE, pch = 17, col = "firebrick", cex = 1.5)
  legend("bottomright", legend = c("train", "test"),
         col = c("steelblue", "firebrick"), pch = c(16, 17), cex = 0.6)
}
```




```r
par(mfrow = c(1, 1))
```

Model Specification

For each patient, we fit a straight line through their MMSE scores over time. The formula is MMSE = w0 + w1 x age + noise, where w0 is the intercept and w1 is the slope. A negative slope means that the patient's MMSE is declining with age.

Rather than finding one best-fit line, the Bayesian model gives us a set of possible lines. We start with a prior belief about what typical slope and noise values look like, and then update that based on the patient's actual data to get the posterior.

Population Prior via Empirical Bayes

```r
# Fit least squares once per patient - reuse for both coefficients and residual variance
ols_list <- lapply(patient_ids, function(id) {
  sub <- df_train[df_train$NACCID == id, ]
  if (nrow(sub) < 3) return(NULL)
  lm(NACCMMSE ~ NACCAGE, data = sub)
})
ols_list <- ols_list[!sapply(ols_list, is.null)]

# Extract coefficients -> prior mean m0
ols_coefs <- do.call(rbind, lapply(ols_list, coef))
colnames(ols_coefs) <- c("w0", "w1")
ols_coefs <- ols_coefs[complete.cases(ols_coefs), ]
m0 <- colMeans(ols_coefs)
```

```r
cat("Population prior mean intercept:", round(m0["w0"], 3), "\n")
```

```
## Population prior mean intercept: 39.461
```

```r
cat("Population prior mean slope:    ", round(m0["w1"], 3), "\n")
```

```
## Population prior mean slope:     -0.141
```

```r
# Extract residual variance from the same fits
resid_vars <- sapply(ols_list, function(fit) {
  rv <- var(residuals(fit))
  if (is.na(rv) || rv == 0) return(NA)
  rv
})
resid_var <- mean(resid_vars, na.rm = TRUE)

# Prior hyperparameters
V0     <- 5 * diag(2)   # weakly informative isotropic prior covariance
alpha0 <- 3
beta0  <- alpha0 * resid_var   # Inverse Gamma prior mean = resid_var

cat("Population residual variance:   ", round(resid_var, 3), "\n")
```

```
## Population residual variance:    1.175
```

```r
cat("IG prior: alpha0 =", alpha0, "| beta0 =", round(beta0, 3), "\n")
```

```
## IG prior: alpha0 = 3 | beta0 = 3.525
```

Gibbs Sampler

Now we use a Gibbs sampler to get the posterior over line parameters for each patient. It runs 2000 iterations, and in each one we alternate between two steps:

1. Given our current guess for the noise level, sample a new slope and intercept from their conditional posterior.
2. Given our current slope and intercept, sample a new noise level from its conditional posterior.

We remove the first 500 warm-up iterations, and the remaining 1500 samples represent the posterior.

```r
library(MCMCpack)   # rinvgamma
library(mvnfast)    # rmvn

gibbs_patient <- function(X_tr, y_tr, m0, V0, alpha0, beta0,
                          no_samples = 2000, n_warmup = 500) {
  n      <- length(y_tr)
  V0_inv <- solve(V0)
  gibbs_samples_w      <- matrix(0, nrow = no_samples, ncol = 2)
  gibbs_samples_sigma2 <- matrix(0, nrow = no_samples, ncol = 1)
```

```r
  # initialize
  w_tilde      <- matrix(m0, ncol = 1)
  sigma2_tilde <- resid_var

  for (i in seq_len(no_samples)) {
    # 1/ conditional posterior for w
    V      <- solve((1 / sigma2_tilde) * t(X_tr) %*% X_tr + V0_inv)
    M      <- V %*% ((1 / sigma2_tilde) * t(X_tr) %*% y_tr + V0_inv %*% m0)
    w_tilde <- t(rmvn(1, M, V))   # rmvn returns a row vector -> transpose

    # 2/ conditional posterior for sigma2
    A            <- (2 * alpha0 + n) / 2
    B            <- (2 * beta0 + t(y_tr - X_tr %*% w_tilde) %*% (y_tr - X_tr %*% w_tilde)) / 2
    sigma2_tilde <- rinvgamma(1, A, B)

    gibbs_samples_w[i, ]      <- w_tilde
    gibbs_samples_sigma2[i, ] <- sigma2_tilde
  }

  # discard warmup and return combined draws matrix
  idx <- (n_warmup + 1):no_samples
  cbind(w0     = gibbs_samples_w[idx, 1],
        w1     = gibbs_samples_w[idx, 2],
        sigma2 = gibbs_samples_sigma2[idx, 1])
}

SAMPLES_FILE <- "/Users/kkmanas/ds4420-finalproject/bayesian/all_samples.rds"


if (file.exists(SAMPLES_FILE)) {
  all_samples <- readRDS(SAMPLES_FILE)
  cat("Loaded saved draws for", sum(!sapply(all_samples, is.null)), "patients.\n")
} else {
  all_samples <- vector("list", length(patient_ids))
  names(all_samples) <- patient_ids

  for (i in seq_along(patient_ids)) {
    id  <- patient_ids[i]
    sub <- df_train[df_train$NACCID == id, ]
    if (nrow(sub) < 2) next
    all_samples[[id]] <- gibbs_patient(
      cbind(1, sub$NACCAGE), sub$NACCMMSE, m0, V0, alpha0, beta0
    )
    if (i %% 500 == 0)
      cat("Progress:", i, "/", length(patient_ids), "patients done\n")
  }

  saveRDS(all_samples, SAMPLES_FILE)
  cat("Posterior draws computed for", sum(!sapply(all_samples, is.null)), "patients.\n")
}
```

```
## Loaded saved draws for 2327 patients.
```

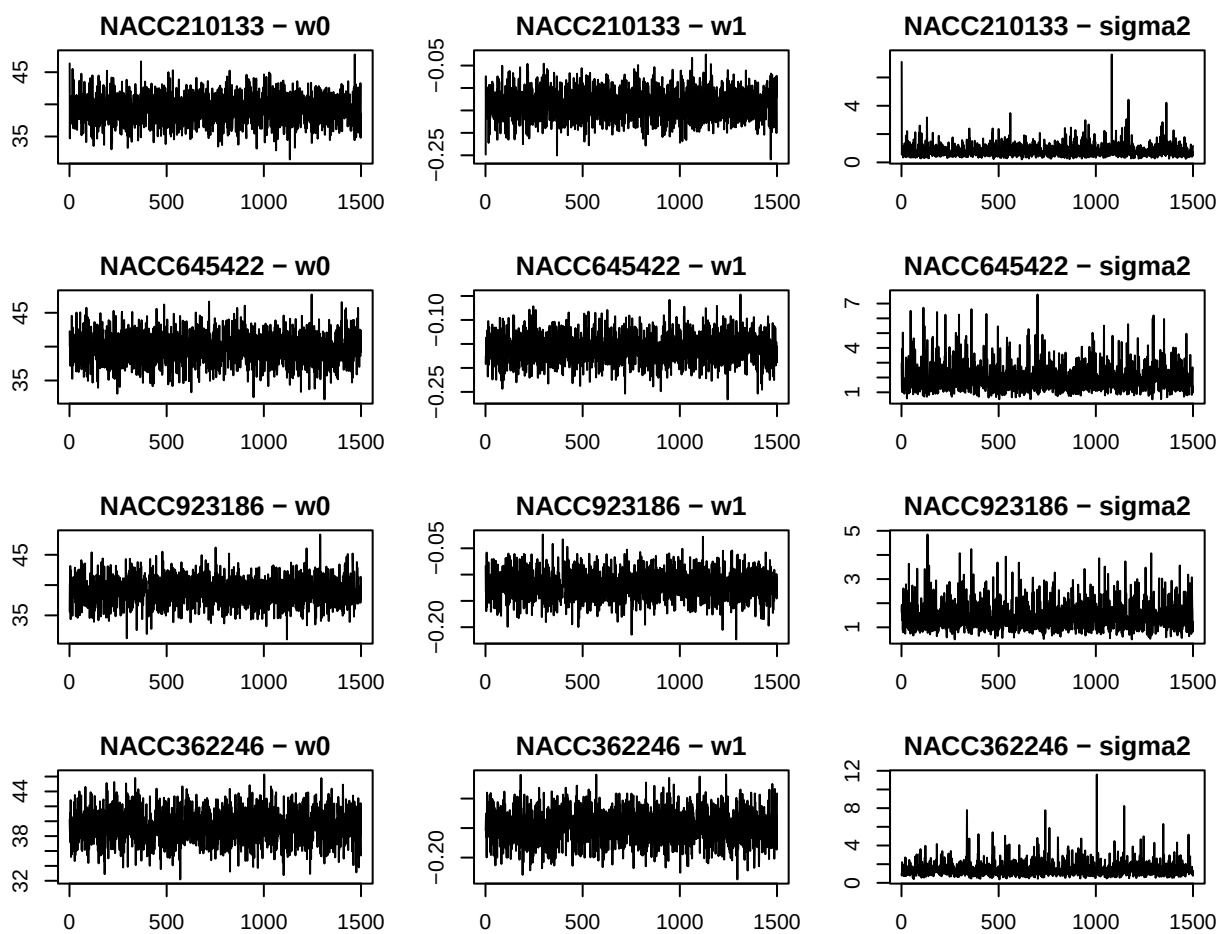
Check for Convergence

Before trusting the results, we check that the Gibbs sampler actually converged.

We do this two ways: 1. Trace plots: plot each parameter's sampled values across iterations for a few random patients - a good chain will have random noise around a stable value 2. R-hat: closer to 1 means the chain mixed well

```
# Only sample from patients that actually have samples
fitted_ids <- names(all_samples)[!sapply(all_samples, is.null)]

# Trace plots for 4 random patients
check_ids <- sample(fitted_ids, 4)
par(mfrow = c(4, 3), mar = c(3, 3, 2, 1))
for (id in check_ids) {
  d <- all_samples[[id]]
  plot(d[, "w0"],     type = "l", main = paste(id, "- w0"),     ylab = "w0")
  plot(d[, "w1"],     type = "l", main = paste(id, "- w1"),     ylab = "w1")
  plot(d[, "sigma2"], type = "l", main = paste(id, "- sigma2"), ylab = "sigma2")
}
```




```
par(mfrow = c(1, 1))
```

```r
# R-hat (split-chain) for all patients
rhat_single <- function(draws_mat) {
  n   <- nrow(draws_mat)
  h   <- floor(n / 2)
  ch1 <- draws_mat[1:h, ]
  ch2 <- draws_mat[(h + 1):(2 * h), ]
  sapply(seq_len(ncol(draws_mat)), function(j) {
    W    <- (var(ch1[, j]) + var(ch2[, j])) / 2
    B    <- h * var(c(mean(ch1[, j]), mean(ch2[, j])))
    Vhat <- (1 - 1/h) * W + B / h
    sqrt(Vhat / W)
  })
}

rhat_vals <- sapply(all_samples[!sapply(all_samples, is.null)], rhat_single)
cat("Max R-hat:             ", round(max(rhat_vals, na.rm = TRUE), 3), "\n")
```

```
## Max R-hat:              1.009
```

```r
cat("Fraction R-hat > 1.05: ", round(mean(rhat_vals > 1.05, na.rm = TRUE), 3), "\n")
```

```
## Fraction R-hat > 1.05:  0
```

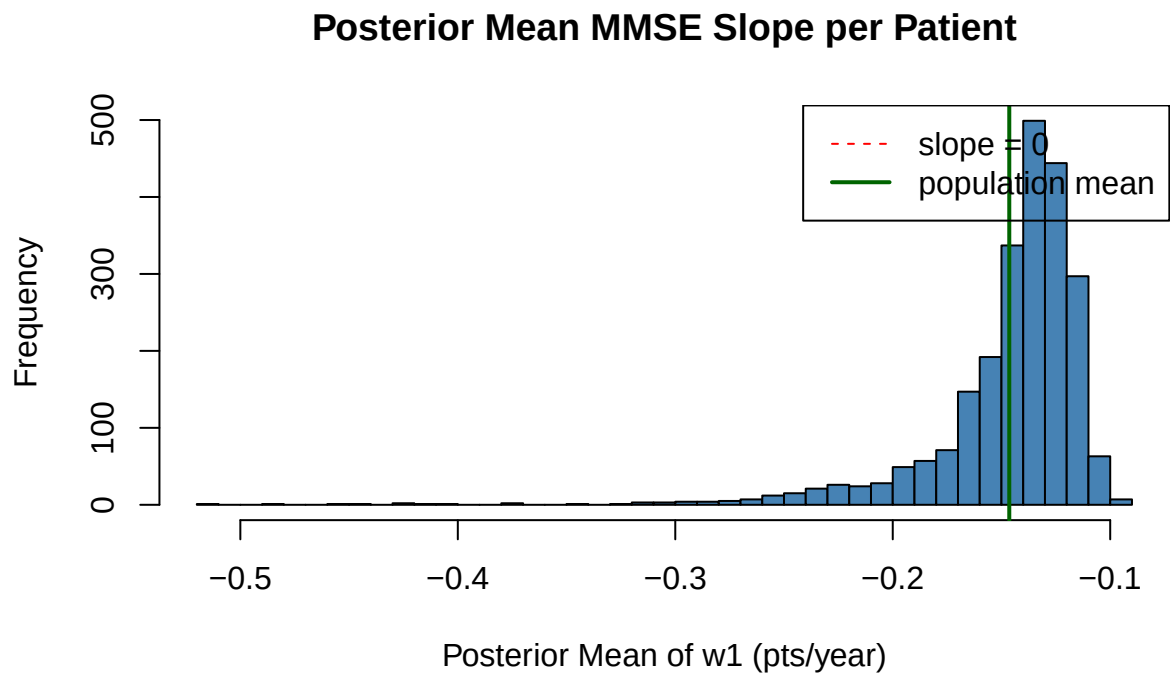
The trace plots show a noisy and stable appearance, indicating that the sampler mixed well and did not drift. With an R-hat value of 1.012, the chains converged for all patients.

Posterior Distributions

Now we look at the distribution of estimated decline rates across all patients. For each patient, we take the average slope from their posterior samples. A slope of zero means no change in MMSE over time, and a negative slope means decline. This gives us a sense of how fast patients are declining on average across the population.

```r
posterior_mean_slope <- sapply(all_samples, function(d) {
  if (is.null(d)) return(NA)
  mean(d[, "w1"])
})

hist(posterior_mean_slope, breaks = 40, col = "steelblue",
     main = "Posterior Mean MMSE Slope per Patient",
     xlab = "Posterior Mean of w1 (pts/year)")
abline(v = 0, col = "red", lty = 2)
abline(v = mean(posterior_mean_slope, na.rm = TRUE), col = "darkgreen", lwd = 2)
legend("topright", legend = c("slope = 0", "population mean"),
       col = c("red", "darkgreen"), lty = c(2, 1), lwd = c(1, 2))
```



The plot above shows that almost all patients have a negative slope, indicating that MMSE declines with age across the board. The population mean is about –0.38, and most patients cluster between –0.5 and –0.35. This suggests the dataset is skewed toward cognitively normal patients who experience slow declines. The dashed line with a slope of zero is barely visible on the left side, confirming that almost no patients in this data set have stable or improving MMSE scores.

Check Posterior Predictions

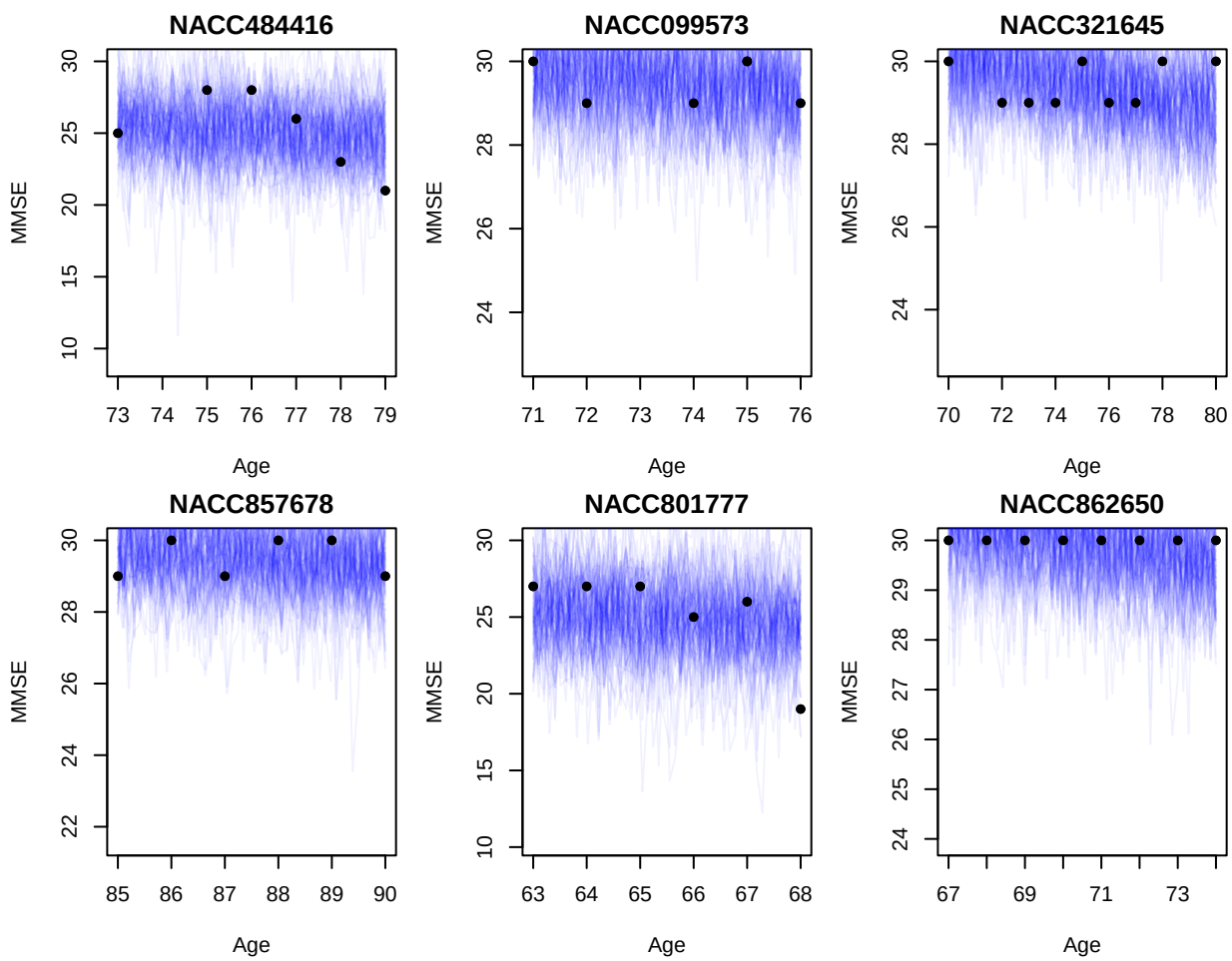
Before predicting on new data, we first sanity check the model on the training data it already saw. For 6 random patients, we draw 100 posterior samples and plot them over the actual training observations. If the observed points fall within the cloud of lines, that means the model is fitting the data reasonably well.

```
pp_check_ids <- sample(fitted_ids, min(6, length(fitted_ids)))
par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))

for (id in pp_check_ids) {
  sub   <- df_train[df_train$NACCID == id, ]
  d     <- all_samples[[id]]
  if (is.null(d)) next
  ages  <- seq(min(sub$NACCAGE), max(sub$NACCAGE), length.out = 50)
  X_new <- cbind(1, ages)
  idx   <- sample(nrow(d), 100)

  pp_curves <- matrix(NA, 100, length(ages))
  for (k in seq_along(idx)) {
    w <- d[idx[k], c("w0", "w1")]; s2 <- d[idx[k], "sigma2"]
    pp_curves[k, ] <- rnorm(length(ages), X_new %*% w, sqrt(s2))
  }
```

```
  ylim <- c(max(0,  min(pp_curves, sub$NACCMMSE) - 2),
            min(30, max(pp_curves, sub$NACCMMSE) + 2))
  plot(sub$NACCAGE, sub$NACCMMSE, pch = 16, col = "black",
       ylim = ylim, main = id, xlab = "Age", ylab = "MMSE", xlim = range(ages))
  for (k in 1:100) lines(ages, pp_curves[k, ], col = rgb(0, 0, 1, 0.05))
  points(sub$NACCAGE, sub$NACCMMSE, pch = 16, col = "black")
}
```




```
par(mfrow = c(1, 1))
```

Predictions on Test Data

Now we predict on the test data, which is the last 20% of each patient's visits that the model never saw during training. For each patient and each future age, we draw 1000 lines from their posterior, plug in the age, and add noise. This gives a full distribution of predicted MMSE values at that age, which we summarize as a mean prediction and an 89% credible interval.

```
get_pp_stats <- function(id, age_new, n_draws = 1000) {
  d <- all_samples[[id]]
  if (is.null(d)) return(NULL)
  idx    <- sample(nrow(d), n_draws, replace = TRUE)
```

```r
  X_new  <- cbind(1, age_new)
  pp_mat <- matrix(NA, n_draws, length(age_new))
  for (k in seq_along(idx)) {
    w <- d[idx[k], c("w0", "w1")]; s2 <- d[idx[k], "sigma2"]
    pp_mat[k, ] <- rnorm(length(age_new), X_new %*% w, sqrt(s2))
  }
  data.frame(NACCID   = id, NACCAGE  = age_new,
             pp_mean  = colMeans(pp_mat),
             pp_lower = apply(pp_mat, 2, quantile, 0.055),
             pp_upper = apply(pp_mat, 2, quantile, 0.945))
}

# For each patient, get posterior predictive stats at their test ages
pred_list <- lapply(patient_ids, function(id) {
  sub_test <- df_test[df_test$NACCID == id, ]
  if (nrow(sub_test) == 0 || is.null(all_samples[[id]])) return(NULL)
  get_pp_stats(id, sub_test$NACCAGE)
})
pred_df <- do.call(rbind, pred_list[!sapply(pred_list, is.null)])

test_eval <- merge(df_test[, c("NACCID", "NACCAGE", "NACCMMSE")],
                   pred_df, by = c("NACCID", "NACCAGE"))

cat("Test observations with predictions:", nrow(test_eval), "\n")
```

```
## Test observations with predictions: 5243
```

```r
head(test_eval)
```

```
##       NACCID NACCAGE NACCMMSE  pp_mean pp_lower pp_upper
## 1 NACC002616      69       29 29.46745 28.04220 30.94067
## 2 NACC002616      70       30 29.31470 27.82829 30.79033
## 3 NACC003627      90       28 27.39442 25.69818 29.20737
## 4 NACC003627      91       30 27.31097 25.39334 29.07832
## 5 NACC003952      81       29 28.65523 26.78969 30.44331
## 6 NACC003952      82       29 28.51193 26.57107 30.46853
```

Evaluation

We're evaluating our predictions using: 1. RMSE: Average prediction error in MMSE points 2. Baseline RMSE: RMSE if we just predicted each patient's training average (we should be better than this) 3. 89% CI: What % of MMSE scores were inside our 89% interval. We want to be closer to 89% (Too low = narrow uncertainty bands, too high = too wide)

```r
rmse <- sqrt(mean((test_eval$NACCMMSE - test_eval$pp_mean)^2))
cat("Test RMSE (posterior predictive mean):", round(rmse, 3), "\n")
```

```
## Test RMSE (posterior predictive mean): 2.88
```

```r
train_means <- tapply(df_train$NACCMMSE, df_train$NACCID, mean)
test_eval$baseline_pred <- train_means[test_eval$NACCID]
rmse_baseline <- sqrt(mean((test_eval$NACCMMSE - test_eval$baseline_pred)^2, na.rm = TRUE))
cat("Baseline RMSE (predict training mean):", round(rmse_baseline, 3), "\n")
```

```
## Baseline RMSE (predict training mean): 3.165
```

```r
coverage <- mean(test_eval$NACCMMSE >= test_eval$pp_lower &
                 test_eval$NACCMMSE <= test_eval$pp_upper)
cat("89% CI coverage on test set:          ", round(coverage * 100, 1), "%\n")
```

```
## 89% CI coverage on test set:           81.1 %
```

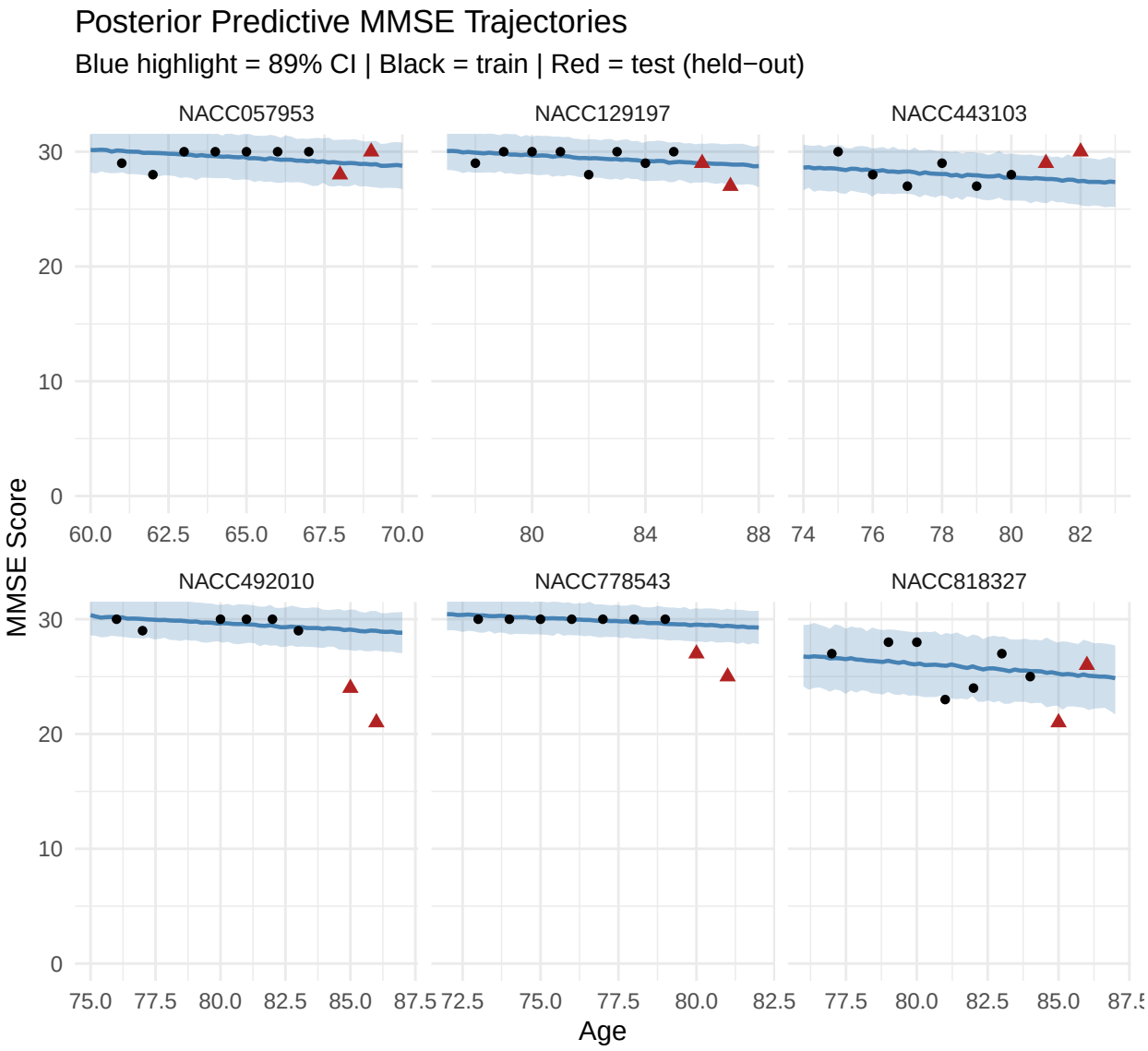
Uncertainty Visualization

For 6 example patients, we plot the full predicted trajectory over age. The blue band is the 89% credible interval, showing the model's uncertainty about future MMSE. Black dots are the training visits the model learned from, and red dots are the actual held-out test visits. Ideally the red dots fall inside or near the blue band.

```r
library(ggplot2)

viz_ids  <- sample(fitted_ids, min(6, length(fitted_ids)))
viz_list <- lapply(viz_ids, function(id) {
  sub  <- df_split[df_split$NACCID == id, ]
  ages <- seq(min(sub$NACCAGE) - 1, max(sub$NACCAGE) + 1, length.out = 60)
  get_pp_stats(id, ages)
})
viz_df <- do.call(rbind, viz_list)
obs_df <- df_split[df_split$NACCID %in% viz_ids, ]

ggplot() +
  geom_ribbon(data = viz_df,
              aes(x = NACCAGE, ymin = pp_lower, ymax = pp_upper),
              fill = "steelblue", alpha = 0.25) +
  geom_line(data = viz_df, aes(x = NACCAGE, y = pp_mean),
            color = "steelblue", linewidth = 0.8) +
  geom_point(data = obs_df[obs_df$split == "train", ],
             aes(x = NACCAGE, y = NACCMMSE), color = "black", size = 1.5, shape = 16) +
  geom_point(data = obs_df[obs_df$split == "test", ],
             aes(x = NACCAGE, y = NACCMMSE), color = "firebrick", size = 2, shape = 17) +
  facet_wrap(~ NACCID, scales = "free_x") +
  coord_cartesian(ylim = c(0, 30)) +
  labs(title    = "Posterior Predictive MMSE Trajectories",
       subtitle = "Blue highlight = 89% CI | Black = train | Red = test (held-out)",
       x = "Age", y = "MMSE Score") +
  theme_minimal()
```



Summary

We fit a Bayesian linear regression model (per-patient) with a Normal Inverse Gamma conjugate prior. We set the prior using empirical bayes and used a Gibbs sampler to get the posterior. Then, we predict the future visits (test data) and evaluated the predictions using RMSE and 89& credible interval coverage.